

一种可扩展的、基于商用器件的数据中心网络架构（A Scalable, Commodity Data Center Network Architecture）

作者：Mohammad Al-Fares（malfares@cs.ucsd.edu）、Alexander Loukissas（aloukiss@cs.ucsd.edu）、Amin Vahdat（vahdat@cs.ucsd.edu）

单位：加州大学圣迭戈分校 计算机科学与工程系（Department of Computer Science and Engineering, University of California, San Diego, La Jolla, CA 92093-0404）

发表：ACM SIGCOMM 2008

摘要（Abstract）

如今的数据中心可能包含数以万计的计算机，对总带宽有着巨大的需求。其网络架构通常由路由与交换设备构成一棵树状结构，越往网络层级的上层，所用设备越专用、越昂贵。遗憾的是，即便部署了最高端的 IP 交换机/路由器，最终得到的拓扑也可能只能支撑网络边缘处可用总带宽的 50%，而成本却极其高昂。数据中心各节点之间带宽的不均匀，使应用设计变得复杂，并限制了整体系统性能。

在本文中，我们展示了如何主要利用商用以太网（commodity Ethernet）交换机，为由数以万计单元构成的集群提供完整的总带宽。正如由商用计算机构成的集群在很大程度上取代了更专用的对称多处理机（SMP）和大规模并行处理机（MPP），我们认为：经过恰当架构与互连的商用交换机，能够以低于当今高端方案的成本提供更高的性能。我们的方法不需要修改端主机的网络接口、操作系统或应用程序；关键在于，它与以太网（Ethernet）、IP 和 TCP 完全向后兼容（backward compatible）。

分类与主题描述符：C.2.1 [网络架构与设计]：网络拓扑；C.2.2 [网络协议]：路由协议

通用术语：设计、性能、管理、可靠性

关键词：数据中心拓扑、等价多路径路由（equal-cost routing）

1. 引言（Introduction）

随着人们对商用 PC 集群运用经验的不断增长，许多机构已经能够以高性价比的方式驾驭千万亿次（petaflop）级别的计算能力和拍字节（petabyte）级别的存储。由数以万计 PC 构成的集群在最大型机构中已不罕见，而千节点级别的集群在大学、研究实验室和公司中也越来越常见。重要的应用类别包括：科学计算、金融分析、数据分析与数据仓库，以及大规模网络服务。

如今，大规模集群中的主要瓶颈往往是节点间通信带宽。许多应用必须与远端节点交换信息，才能继续其本地计算。例如，MapReduce 必须执行大量的数据洗牌（data shuffling），以便在进入 reduce 阶段之前传输其 map 阶段的输出。运行在基于集群的文件系统上的应用，通常在进行 I/O 操作前需要访问远端节点。一次对网络搜索引擎的查询，往往需要与集群中托管倒排索引的每一个节点进行并行通信，以返回最相关的结果。即便在逻辑上彼此独立的集群之间，也常常存在显著的通信需求，例如

当负责构建索引的站点为各个执行搜索的集群更新倒排索引时。互联网服务越来越多地采用面向服务的架构（service oriented architecture），其中检索一个网页都可能需要与运行在远端节点上的数百个独立子服务进行协调与通信。最后，并行科学应用对通信的巨大需求也是众所周知的。

为大规模集群构建通信结构（communication fabric），在宏观层面上有两种选择。一种选择是利用专用硬件和通信协议，例如 InfiniBand 或 Myrinet。尽管这些方案可以扩展到数千节点的集群并提供高带宽，但它们不利用商用器件（因此更昂贵），并且无法原生兼容 TCP/IP 应用。第二种选择是利用商用以太网交换机和路由器来互连集群机器。这种方法支持人们熟悉的管理基础设施，并能运行未经修改的应用、操作系统和硬件。遗憾的是，集群的总带宽随集群规模的扩展性很差，而要达到最高水平的带宽，则会随集群规模产生非线性的成本增长。

出于兼容性和成本方面的考虑，大多数集群通信系统采用第二种方法。然而，在大型集群中，通信带宽可能会因通信模式的不同而出现显著的过载（oversubscription）。也就是说，连接到同一台物理交换机上的两个节点也许能够以全带宽（例如 1 Gbps）通信，但跨越交换机进行通信——可能要跨越层级结构中的多个层次——则可能会严重限制可用带宽。要解决这些瓶颈，需要非商用的方案，例如大型 10 Gbps 交换机和路由器。此外，在互连交换机构成的树状结构上典型的单路径路由意味着，整个集群的带宽受限于通信层级根部所能提供的带宽。即使我们正处于 10 Gbps 技术开始具备成本竞争力的转折点，最大的 10 Gbps 交换机仍然成本高昂，并且对于最大型的集群而言仍会限制整体可用带宽。

在此背景下，本文的目标是设计一种满足以下目标的数据中心通信架构：

- 可扩展的互连带宽（Scalable interconnection bandwidth）：数据中心中的任意一台主机都应当能够以其本地网络接口的全带宽，与网络中的任意其他主机通信。
- 规模经济（Economies of scale）：正如商用个人计算机成为大规模计算环境的基础，我们希望利用同样的规模经济，让廉价的现成（off-the-shelf）以太网交换机成为大规模数据中心网络的基础。
- 向后兼容（Backward compatibility）：整个系统应当与运行以太网和 IP 的主机向后兼容。也就是说，现有的数据中心——它们几乎普遍采用商用以太网并运行 IP——应当能够在不做任何修改的情况下利用这种新的互连架构。

我们证明：通过以胖树（fat-tree）架构互连商用交换机，我们能够为由数以万计节点构成的集群实现完整的双边带宽（bisection bandwidth）。具体而言，我们架构的一个实例采用 48 端口的以太网交换机，能够为多达 27648 台主机提供全带宽。通过严格使用商用交换机，我们在提供更高带宽的同时实现了比现有方案更低的成本。我们的方案不需要改动端主机，完全兼容 TCP/IP，并且只需对交换机本身的转发功能做适度修改。我们还预期，一旦 10 GigE 交换机在网络边缘成为商用器件，鉴于当前缺乏任何更高速的以太网替代方案（无论成本如何），我们的方法将是为大型集群提供全带宽的唯一途径。即便将来出现更高速的以太网方案，它们初期也将只有较小的端口密度，且成本高昂。

2. 背景（Background）

2.1 当前的数据中心网络拓扑（Current Data Center Network Topologies）

我们开展了一项研究，以确定当前数据中心通信网络的最佳实践。这里我们聚焦于利用以太网和 IP 的商用设计；我们将在第 7 节讨论本工作与替代技术之间的关系。

2.1.1 拓扑 (Topology)

如今典型的架构由两层或三层的交换机/路由器树构成。三层设计（见图 1）在树根有一个核心层（core tier），中间是汇聚层（aggregation tier），在树叶处是边缘层（edge tier）。两层设计则只有核心层和边缘层。通常，两层设计可支持 5K 到 8K 台主机。由于我们的目标约为 25000 台主机，因此我们将注意力限定在三层设计上。

树叶处的交换机（脚注：本文余下部分用“交换机”一词泛指同时执行二层交换和三层路由的设备）拥有若干 GigE 端口（48 – 288 个），以及若干 10 GigE 上行链路，连接到一层或多层在叶交换机之间汇聚并转发分组的网络设备。在层级结构的更高层，是带有 10 GigE 端口（通常 32 – 128 个）和巨大交换容量的交换机，用于在各边缘之间汇聚流量。

我们假设使用两类交换机，它们代表了当前在端口密度和带宽两方面的高端水平。第一类用于树的边缘，是一款 48 端口 GigE 交换机，带有四个 10 GigE 上行链路。对于通信层级的更高层，我们考虑 128 端口的 10 GigE 交换机。两类交换机都允许所有直连主机以其网络接口的全速彼此通信。

> 图 1：常见的数据中心互连拓扑。主机到交换机的链路为 GigE，交换机之间的链路为 10 GigE。图中自上而下分别为核心层（Core）、汇聚层（Aggregation）、边缘层（Edge）的多根树结构。

2.1.2 过载 (Oversubscription)

许多数据中心设计引入过载（oversubscription）作为降低设计总成本的手段。我们将“过载比”一词定义为：在某一特定通信拓扑中，端主机之间最坏情况下可达到的总带宽与该拓扑总双边带宽（bisection bandwidth）之比。1:1

的过载比表示所有主机都有可能以其网络接口的全带宽（例如商用以太网设计中的 1 Gb/s）与任意其他主机通信。5:1 的过载值则意味着，对于某些通信模式，仅有 20% 的主机可用带宽可被使用。典型设计的过载比为 2.5:1（400 Mbps）到 8:1（125 Mbps）。尽管对于 1 Gb/s 以太网而言，过载比为 1:1 的数据中心是可行的，但正如我们在第 2.1.4 节所讨论的，此类设计的成本通常高得令人望而却步，即便对于中等规模的数据中心也是如此。对于 10 Gb/s 以太网，一旦超出单台交换机的范围，目前还无法实现完整的双边带宽。

2.1.3 多路径路由 (Multi-path Routing)

要在更大的集群中实现任意主机之间的全带宽，需要一棵带有多个核心交换机的“多根（multi-rooted）”树（见图 1）。这又需要一种多路径路由技术，例如 ECMP（Equal-Cost Multipath，等价多路径）。目前，大多数企业级核心交换机支持 ECMP。如果不使用 ECMP，那么以单根核心、1:1 过载比所能支持的最大集群将被限制在 1280 个节点（对应于单台 128 端口 10 GigE 交换机所提供的带宽）。

为了利用多条路径，ECMP 在各条流之间执行静态的负载分摊（load splitting）。它在做分配决策时不考虑流的带宽，这可能导致即便对简单的通信模式也出现过载。此外，当前的 ECMP 实现将路径的多重性限制为 8 – 16 条，这往往低于为更大型数据中心提供高双边带宽所需的路径多样性。另外，路由表条目数量会随所考虑路径数的增加而成倍增长，这既增加了成本，也会增加查找延迟。

2.1.4 成本（Cost）

为大型集群构建网络互连的成本极大地影响着设计决策。正如我们上面所讨论的，引入过载通常是为了降低总成本。这里我们采用当前的最佳实践，给出不同主机数量和过载比下各种配置的粗略成本。我们假设边缘处每台 48 端口 GigE 交换机的成本为 7000 美元，汇聚层和核心层每台 128 端口 10 GigE 交换机的成本为 700000 美元。这些计算中不考虑布线成本。

图 2 以 x

轴上端主机的总数为自变量，绘制出以百万美元计的成本。每条曲线代表一个目标过载比。例如，将 20000 台主机以彼此间全带宽互连所需的交换硬件成本约为 3700 万美元。对应于 3:1 过载比的曲线，绘制的是将端主机互连且任意端主机通信的最大可用带宽被限制在约 330 Mbps 时的成本。我们还把使用我们所提出的胖树架构实现 1:1 过载比的成本一并列入，以作比较。

> 图 2：在不同过载比下，当前的成本估算与可支持的最大主机数之间的关系。曲线包括 1:1、3:1、7:1 的传统层级设计，以及胖树（Fat-tree）方案；胖树在大规模下成本显著更低。

总体而言，我们发现现有的在大型集群中提供高带宽的技术都会带来巨大成本，而基于胖树的集群互连在以适中成本提供可扩展带宽方面展现出巨大潜力。然而，从某种意义上说，图 2 低估了在构建数据中心架构时采用最高端组件的难度和开销。2008 年，10 GigE 交换机正处于即将成为商用器件的边缘；以"每端口每比特/秒"价格来比较 GigE 和 10 GigE 交换机，二者大约有 5 倍的差距，而这一差距还在继续缩小。为考察历史趋势，我们在表 1 中给出了在某一特定年份、使用当年可获得的最高端交换机所能支持的最大集群配置的成本。我们的取值基于对 2002、2004、2006 和 2008 年各厂商高端 10 GigE 交换机产品发布的历史研究。

我们利用这些发现，在保持 1:1 过载比的前提下，构建当年技术所能支持的最大集群配置。表 1 给出了某一特定年份可获得的最大 10 GigE 交换机；在层级化设计中，我们将其用于核心层和汇聚层。表 1 也给出了当年可获得的最大商用 GigE 交换机；在胖树的所有层级以及层级化设计的边缘层，我们都采用这些交换机。

> 表 1：不同年份在 1:1 过载比下可支持的最大集群规模。

>

> | 年份 | 最大 10 GigE 交换机 | 层级化设计 主机数 | 层级化设计 每 GigE 成本 | 最大 GigE 交换机 | 胖树 主机数 | 胖树 每 GigE 成本 |

> | --- | --- | --- | --- | --- | --- | --- |

> | 2002 | 28 端口 | 4,480 | \$25.3K | 28 端口 | 5,488 | \$4.5K |

> | 2004 | 32 端口 | 7,680 | \$4.4K | 48 端口 | 27,648 | \$1.6K |

> | 2006 | 64 端口 | 10,240 | \$2.1K | 48 端口 | 27,648 | \$1.2K |

> | 2008 | 128 端口 | 20,480 | \$1.8K | 48 端口 | 27,648 | \$0.3K |

直到最近，传统技术中采用高端交换机所支持的最大集群规模一直受限于可用的端口密度。此外，在 10 GigE 交换机刚出现时，高端交换机的成本高得令人却步。需要注意的是，我们对传统层级结构的计算其实是相当宽容的，因为商用 GigE 交换机在汇聚层直到最近才具备所需的 10 GigE 上行链路。另一方面，基于胖树拓扑的集群扩展性良好，其总成本下降得更快、更早（这是因为它更早地遵循了商用

器件的价格趋势)。同时,胖树拓扑也不需要更高速的上行链路。

最后,有趣的是要指出:今天,要构建一个所有节点之间都有可能获得 10 Gbps 带宽的 27648 节点集群,在技术上是不可行的。另一方面,胖树交换架构可以利用接近商用水平的 48 端口 10 GigE 交换机,成本将超过 6.9 亿美元。尽管在大多数场景下这一成本很可能高得难以承受,但归根结底的关键在于:使用传统的、采用高端交换机的汇聚方式,甚至根本无法构建出这样的配置,因为当今市面上既没有比 10 GigE 更快的交换机产品,甚至连相应的以太网标准都没有。

2.2 Clos 网络/胖树 (Clos Networks/Fat-Trees)

如今,商用交换机与非商用交换机之间的价格差距,为人们提供了强烈的动机,去用许多小型商用交换机、而非更少的大型且更昂贵的交换机来构建大规模通信网络。五十多年前,电话交换机领域类似的趋势促使 Charles Clos

设计出一种网络拓扑:通过恰当地互连较小的商用交换机,为众多端设备提供高水平带宽。

我们采用 Clos 拓扑的一个特例——称为胖树 (fat-tree)——来互连商用以太网交换机。我们将一个 k 元 (k -ary) 胖树组织成如图 3 所示的结构。其中有 k 个荚舱 (pod), 每个荚舱包含两层, 每层 $k/2$ 台交换机。下层中的每台 k 端口交换机直接连接 $k/2$ 台主机。其余 $k/2$ 个端口中的每一个, 连接到该层级结构汇聚层 k 个端口中的 $k/2$ 个。

共有 $(k/2)^2$ 台 k 端口核心交换机。每台核心交换机都有一个端口连接到 k 个荚舱中的每一个。任意核心交换机的第 i 个端口连接到第 i 个荚舱, 使得每个荚舱交换机汇聚层中相邻的端口以 $(k/2)$ 为步长 (stride) 连接到核心交换机。一般而言, 用 k 端口交换机构建的胖树可支持 $k^3/4$ 台主机。本文中我们聚焦于 $k \leq 48$ 的设计。我们的方法可推广到任意 k 值。

胖树拓扑的一个优势是:所有交换单元都是相同的, 使我们能够在通信架构中的所有交换机上利用廉价的商用器件(脚注:注意并不要求交换机同构, 核心处可使用更大的交换机, 例如用于复用。虽然这类交换机的平均无故障时间 MTTF 可能更长, 但这会抵消成本优势, 并且布线开销不变)。此外, 胖树是可重排无阻塞 (rearrangeably non-blocking) 的, 这意味着对于任意通信模式, 总存在某一组路径能够饱和拓扑中端主机所能获得的全部带宽。在实践中要达到 1:1 的过载比可能很困难, 因为需要防止 TCP 流的分组乱序。

图 3 展示了 $k=4$ 的胖树这一最简单的非平凡实例。连接到同一台边缘交换机的所有主机构成它们自己的子网。因此, 发往连接到同一台下层交换机的主机的所有流量都是被"交换" (switched) 的, 而所有其他流量都是被"路由" (routed) 的。作为该拓扑的一个示例实例, 用 48 端口 GigE 交换机构建的胖树将由 48 个荚舱构成, 每个荚舱包含一个边缘层和一个汇聚层, 每层各 24 台交换机。每个荚舱中的边缘交换机各分配 24 台主机。该网络支持 27648 台主机, 由 1152 个子网构成, 每个子网含 24 台主机。在不同荚舱中任意一对主机之间存在 576 条等价路径。部署这样一种网络架构的成本约为 864 万美元, 相比之下, 前述传统技术的成本约为 3700 万美元。

> 图 3: 简单的胖树拓扑 ($k=4$)。使用第 3.3 节描述的两层路由表, 从源 10.0.1.2 发往目的 10.2.0.3 的分组将沿虚线所示路径传输。图中标注了各层交换机的地址, 如核心层 10.4.1.1、汇聚层 10.0.2.1、边缘层 10.0.1.1 等。

2.3 小结 (Summary)

给定我们的目标网络架构，在本文余下部分，我们将解决在以太网部署中采用该拓扑的两个主要问题。第一，IP/以太网网络通常在每对源和目的之间只构建单一路由路径。即便对于简单的通信模式，这种单路径路由也会很快在胖树上下方向上造成瓶颈，从而显著限制整体性能。我们将描述对 IP 转发的一些简单扩展，以有效利用胖树所提供的高扇出 (fan-out)。第二，在大型网络中，胖树拓扑可能带来显著的布线复杂度。在某种程度上，这种开销是胖树拓扑所固有的，但在第 6 节中，我们将提出封装与摆放技术来缓解这种开销。最后，我们已在 Click 中构建了我们架构的一个原型，详见第 3 节。第 5 节呈现的初步性能评估，证实了我们的方法在小规模部署中潜在的性能优势。

3. 架构 (Architecture)

在本节中，我们描述一种以胖树拓扑互连商用交换机的架构。我们首先阐述对路由表结构做轻微修改的必要性。然后，我们描述如何为集群中的主机分配 IP 地址。接着，我们引入两层路由查找 (two-level route lookup) 的概念，以协助胖树上的多路径路由。随后，我们呈现用于填充每台交换机转发表的算法。我们还描述了作为替代性多路径路由方法的流分类 (flow classification) 和流调度 (flow scheduling) 技术。最后，我们呈现一种简单的容错 (fault-tolerance) 方案，并描述我们方法的散热与功耗特性。

3.1 动机 (Motivation)

要在该网络中实现最大的双边带宽，需要将任意给定荚舱的出向流量尽可能均匀地分散到各个核心交换机上。诸如 OSPF2 之类的路由协议通常以跳数 (hop-count) 作为“最短路径”的度量，而在 k 元胖树拓扑中 (见第 2.2 节)，任意两个位于不同荚舱的主机之间存在 $(k/2)^2$ 条这样的最短路径，但只有一条会被选中。因此，即便存在代价相同的其他选择，交换机也会把发往某给定子网的流量集中到单个端口。此外，取决于 OSPF 消息到达时刻的交错情况，有可能只有极少数核心交换机——甚至可能只有一台——被选作荚舱间的中间链路。这会在这些点上造成严重拥塞，并且无法利用胖树中的路径冗余。

诸如 OSPF-ECMP 之类的扩展，除了在我们所考虑的这类交换机中不可用之外，还会导致所需前缀数量的爆炸式增长。一台下层荚舱交换机将需要为每一个其他子网设置 $(k/2)$ 个前缀；总计为 $k * (k/2)^2$ 个前缀。

因此，我们需要一种简单的、细粒度的荚舱间流量扩散方法，它能利用该拓扑的结构。交换机必须能够识别需要被均匀分散的那一类流量，并对其予以特殊处理。为实现这一点，我们提议使用两层路由表 (two-level routing table)，它根据目的 IP 地址的低位比特来分散出向流量 (见第 3.3 节)。

3.2 寻址 (Addressing)

我们将网络中所有 IP 地址都分配在私有的 10.0.0.0/8 地址块内。我们遵循人们熟悉的四点分十进制形式，并满足以下条件：荚舱交换机被赋予 10.pod.switch.1 形式的地址，其中 pod 表示荚舱编号 (取值范围 $[0, k - 1]$)，switch 表示该交换机在荚舱中的位置 (取值范围 $[0, k - 1]$ ，从左到右、自下而上)。我们为核心交换机赋予

10.k.j.i 形式的地址，其中 j 和 i 表示该交换机在 $(k/2)^2$ 核心交换机网格中的坐标（各自取值范围 $[1, (k/2)]$ ，从左上角开始）。

主机的地址由它所连接的荚舱交换机推导而来；主机的地址形如 10.pod.switch.ID，其中 ID 是该主机在其子网中的位置（取值范围 $[2, k/2 + 1]$ ，从左到右）。因此，每台下层交换机负责一个 $/24$ 子网，其中含 $k/2$ 台主机（当 $k < 256$ 时）。图 3 展示了 $k = 4$ 的胖树所对应的这种寻址方案的示例。尽管这种方式对可用地址空间的利用相对浪费，但它简化了路由表的构建，如下文所述。尽管如此，该方案可扩展到多达 420 万台主机。

3.3 两层路由表（Two-Level Routing Table）

为提供第 3.1 节所阐述的均匀分布机制，我们修改路由表，以支持两层前缀查找。主路由表中的每个条目都可能附带一个指向小型二级表的额外指针，该二级表由（后缀，端口）条目构成。如果一个一级前缀不包含任何二级后缀，则称其为终止性（terminating）前缀；而一张二级表可以被不止一个一级前缀所指向。主表中的条目是左对齐（left-handed）的（即 $/m$ 前缀掩码形如

$1\ m\ 0^{\ 3\ 2\ \dots\ m}$ ），而二级表中的条目是右对齐（right-handed）的（即 $/m$ 后缀掩码形如 $0^{\ 3\ 2\ \dots\ m}\ 1\ m$ ）。

。如果最长匹配前缀查找得到的是一个非终止性前缀，那么就在二级表中查找最长匹配后缀并使用之。

这种两层结构会略微增加路由表的查找延迟，但前缀查找在硬件中的并行特性应能确保只带来微小的代价（见下文）。这一点还得益于这些表本就被设计得非常小。如下文所示，任意荚舱交换机的路由表所含前缀不超过 $k/2$ 个，后缀不超过 $k/2$ 个。

> 图 4：两层表示例。这是交换机 10.2.2.1 上的表。一个目的 IP 地址为 10.2.1.2 的入向分组会在端口 1 转发；而一个目的 IP 地址为 10.3.0.3 的分组会在端口 3 转发。

> 一级前缀表：10.2.0.0/24 → 端口 0；10.2.1.0/24 → 端口 1；0.0.0.0/0 → 指向二级后缀表。

> 二级后缀表：0.0.0.2/8 → 端口 2；0.0.0.3/8 → 端口 3。

3.4 两层查找的硬件实现（Two-Level Lookup Implementation）

下面我们描述如何使用内容寻址存储器（Content-Addressable Memory, CAM）在硬件中实现这种两层查找。CAM

用于搜索密集型的应用，在针对某一比特模式进行匹配查找时，比算法式方法更快。CAM 能够在单个时钟周期内对其所有条目进行并行搜索。查找引擎使用一种特殊的 CAM，称为三态 CAM（Ternary CAM, TCAM）。除了在特定位上匹配 0 和 1 之外，TCAM 还能存储“无关位（don't care）”，这使它适合存储变长前缀，例如路由表中那些前缀。其缺点在于：CAM 的存储密度相当低，非常耗电，且单比特成本高。然而，在我们的架构中，路由表可以用相对适中规模的 TCAM 来实现（ k 个条目，每个 32 比特宽）。

图 5 展示了我们所提议的两层查找引擎实现。一个 TCAM 存储地址前缀和后缀，它进而索引一个 RAM，RAM 中存储下一跳的 IP 地址和输出端口。我们将左对齐（前缀）条目存放在数值较小的地址处，将右对齐（后缀）条目存放在较大的地址处。我们对 CAM 的输出进行编码，使得匹配地址中数值最小的那个条目被输出。这恰好满足了我们这一两层查找的特定语义：当一个分组的目的 IP 地址同时匹配一个左对齐条目和一个右对齐条目时，应选择左对齐条目。例如，使用图 5 中的路由表，一个目的 IP 地址为 10.2.0.3 的分组同时匹配左对齐条目 10.2.0.X 和右对齐条目

X.X.X.3。该分组会被正确地在端口 0 转发。然而，一个目的 IP 地址为 10.3.1.2 的分组只匹配右对齐条目 X.X.X.2，会在端口 2 转发。

> 图 5：TCAM 两层路由表实现。TCAM 中存储 10.2.0.X、10.2.1.X、X.X.X.2、X.X.X.3

等条目，经编码器后索引到 RAM，RAM 中给出下一跳（如

10.2.0.1、10.2.1.1、10.4.1.1、10.4.1.2）和输出端口（0、1、2、3）。

3.5 路由算法（Routing Algorithm）

胖树中前两层的交换机充当过滤式的流量扩散器（filtering traffic diffusers）；任意给定荚舱中的下层和上层交换机，对该荚舱内的各子网都拥有终止性前缀。因此，如果一台主机向同一荚舱内但不同子网的另一台主机发送分组，那么该荚舱中所有上层交换机都会有一个指向目的子网交换机的终止性前缀。

对于所有其他出向的荚舱间流量，荚舱交换机拥有一个默认的 /0 前缀，并配有一张匹配主机 ID（目的 IP 地址的最低有效字节）的二级表。我们将主机 ID 用作一种确定性熵（deterministic entropy）的来源；它们会使流量在通往核心交换机的各条出向链路上向上均匀分散（脚注：由于这些表是静态的，有可能达不到完美的分布。我们将在第 5 节考察最坏情况的通信模式）。这也会使发往同一主机的后续分组沿同一路径传输，从而避免分组乱序。

在核心交换机中，我们为所有网络 ID 分配终止性的一级前缀，每个前缀指向包含该网络的相应荚舱。一旦分组到达某台核心交换机，到其目的荚舱就只有恰好一条链路，该交换机将为该分组的荚舱包含一个终止性的 /16 前缀（10.pod.0.0/16, port）。一旦分组到达其目的荚舱，接收它的上层荚舱交换机也会包含一个（10.pod.switch.0/24, port）前缀，将该分组导向其目的子网交换机，在那里它最终被交换到目的主机。因此，流量扩散只发生在分组旅程的前半段。

可以设计分布式协议，在每台交换机中增量地构建所需的转发状态。然而为简单起见，我们假设有一个对集群互连拓扑拥有完整知识的中心实体。这个中心化的路由控制负责静态地生成所有路由表，并在网络建立阶段将这些表加载到交换机中。动态路由协议还将负责检测各个交换机的故障并执行路径故障切换（path fail-over，见第 3.8 节）。下面，我们总结在荚舱交换机和核心交换机两处生成转发表步骤。

荚舱交换机（Pod Switches）。

在每台荚舱交换机中，我们为荚舱内所含的各子网分配终止性前缀。对于荚舱间流量，我们添加一个 /0 前缀，并配以一张匹配主机 ID 的二级表。算法 1

给出了为上层荚舱交换机生成路由表的伪代码。在出向端口上做取模移位（modulo shift）的原因，是为了避免来自不同下层交换机、却发往同一主机 ID

的流量都流向同一台上层交换机。对于下层荚舱交换机，我们只需省略第 3 行中的 /24 子网前缀步骤，因为该子网自身的流量是被交换的，而荚舱内与荚舱间流量都应在各上层交换机之间均匀分摊。

```
1 foreach pod x in [0, k - 1] do           // 遍历每个荚舱
2   foreach switch z in [(k/2), k - 1] do   // 遍历该荚舱中的上层交换机
3     foreach subnet i in [0, (k/2) - 1] do // 遍历荚舱内每个子网
4       addPrefix(10.x.z.1, 10.x.i.0/24, i); // 为本荚舱子网添加终止性前缀，指向端口 i
5     end
6   addPrefix(10.x.z.1, 0.0.0.0/0, 0);      // 添加默认 /0 前缀（用于荚舱间流量）
7   foreach host ID i in [2, (k/2) + 1] do  // 遍历主机 ID
```



```

8      addSuffix(10.x.z.1, 0.0.0.i/8,
      (i - 2 + z) mod (k/2) + (k/2));    // 按主机 ID 在上行端口间做取模移位分散
9  end
10 end
11 end

```

> 算法 1：生成汇聚（上层荚舱）交换机的路由表。假设函数签名为 addPrefix(switch, prefix, port)、addSuffix(switch, suffix, port)，其中 addSuffix 会把一个二级后缀添加到最近添加的那个一级前缀之下。

核心交换机（Core Switches）。

由于每台核心交换机都连接到每个荚舱（端口 i 连接到荚舱 i ），核心交换机只包含指向其目的荚舱的终止性 /16 前缀，如算法 2 所示。该算法生成的表，其大小与 k 成线性关系。网络中没有任何交换机的表会包含超过 k 个一级前缀或 $k/2$ 个二级后缀。

```

1 foreach j in [1, (k/2)] do           // 遍历核心交换机网格的行坐标 j
2   foreach i in [1, (k/2)] do         // 遍历核心交换机网格的列坐标 i
3     foreach destination pod x in [0, (k/2) - 1] do // 遍历每个目的荚舱
4       addPrefix(10.k.j.i, 10.x.0.0/16, x); // 为目的荚舱 x 添加终止性 /16 前缀，指向端口 x
5     end
6   end
7 end

```

> 算法 2：生成核心交换机的路由表。

路由示例（Routing Example）。

为说明使用两层表时的网络运作，我们给出图 3 所示从源 10.0.1.2 到目的 10.2.0.3 的分组所做路由决策的示例。首先，源主机的网关交换机（10.0.1.1）只会将该分组与 /0 一级前缀匹配，因此会根据该前缀的二级表、依据主机 ID 字节来转发该分组。在那张表中，该分组匹配 0.0.0.3/8 后缀，指向端口 2 和交换机 10.0.2.1。交换机 10.0.2.1 同样遵循相同步骤，在端口 3 转发，连接到核心交换机 10.4.1.1。该核心交换机将分组匹配到一个终止性的 10.2.0.0/16 前缀，它在端口 2 指向目的荚舱 2 和交换机 10.2.2.1。这台交换机与目的子网属于同一荚舱，因此拥有一个终止性前缀 10.2.0.0/24，它在端口 0 指向负责该子网的交换机 10.2.0.1。从那里开始，标准的交换技术将分组送达目的主机 10.2.0.3。

请注意，对于从 10.0.1.3 到另一台主机 10.2.0.2 的同时通信，传统的单路径 IP 路由会沿与上述流相同的路径传输，因为两个目的都在同一子网上。遗憾的是，这会消除胖树拓扑的所有扇出优势。相反，我们的两层表查找允许交换机 10.0.1.1 基于两层表中的右对齐匹配，将第二条流转发到 10.0.3.1。

3.6 流分类（Flow Classification）

除了上面描述的两层路由技术外，我们还考虑了两种可选的动态路由技术，因为它们目前已在若干商用路由器中可用。我们的目标是量化这些技术潜在的好处，但同时承认它们会带来额外的每分组开销。重要的是，这些方案中所维护的任何状态都是软状态（soft state），各个交换机在状态丢失时可以回退到两层路由。

作为一种向核心交换机扩散流量的替代方法，我们在荚舱交换机中执行带有动态端口重分配（dynamic port-reassignment）的流分类，以克服那些本可避免的局部拥塞情形（例如，当两条流竞争同一输出端口时，即便存在另一个到目的代价相同、却利用不足的端口）。我们将一条流（flow）定义为：在分组

首部某些字段子集（通常是源和目的 IP 地址、目的传输层端口）上具有相同取值的一系列分组。具体而言，荚舱交换机会：

1. 识别同一条流的后续分组，并将它们在同一出向端口上转发。
2. 周期性地重分配数量最少的若干流的输出端口，以尽量减小不同端口之间总流量容量的差异。

步骤 1 是一种防止分组乱序的措施，而步骤 2

旨在面对动态变化的流大小时，确保流在向上指向的各端口上公平分布。第 4.2

节将更详细地描述我们对流分类器的实现及其流分布启发式方法。

3.7 流调度 (Flow Scheduling)

若干研究表明，互联网流量的传输时间和突发长度的分布是长尾的 (long-tailed)，其特征是：少数大型、长寿命的流（负责大部分带宽）与众多小型、短寿命的流并存。我们认为，对大流的路由在决定网络可达双边带宽方面起着最重要的作用，因此值得特殊处理。在这种流管理的替代方法中，我们对大流进行调度，使它们彼此之间的重叠最小化。这一选择由一个中央调度器 (central scheduler) 做出，它对网络中所有活跃的大流拥有全局知识。在这个初始设计中，我们只考虑每台主机同一时刻只发起一条大流的情形。

3.7.1 边缘交换机 (Edge Switches)

如前所述，边缘交换机起初会在本地将一条新流分配给负载最轻的端口。然而，边缘交换机还会额外检测任何大小增长超过预定义阈值的出向流，并周期性地向中央调度器发送通知，指明所有活跃大流的源和目的。这相当于边缘交换机请求将该流放置到一条无争用 (uncontended) 的路径上。

请注意，与第 3.6 节不同，本方案不允许边缘交换机独立地重分配一条流的端口，无论流的大小如何。中央调度器是唯一有权下达重分配命令的实体。

3.7.2 中央调度器 (Central Scheduler)

中央调度器（可能有副本）跟踪所有活跃的大流，并尽可能为它们分配互不冲突的路径。调度器为网络中所有链路维护布尔状态，表示它们是否可用于承载大流。

对于荚舱间流量，回想一下，网络中任意一对主机之间存在 $(k/2)^2$ 条可能的路径，而每条这样的路径对应一台核心交换机。当调度器收到一条新流的通知时，它会在各核心交换机之间线性搜索，以找到一台其对应路径组件不包含任何已预留链路的核心交换机（脚注：要为所有大流找到最优放置，要么需要提前知道所有流的源和目的，要么需要对已有流进行路径重分配；不过这种贪心启发式方法给出了良好的近似，在仿真中对 27k 台主机间随机目的的流可达到 94% 的效率）。一旦找到这样一条路径，调度器就将这些链路标记为已预留，并以该流所选路径对应的正确出向端口，通知源荚舱中相关的下层和上层交换机。对于荚舱内的大流，会执行类似的搜索；这一次是在某台上层荚舱交换机中寻找一条无争用的路径。调度器会对那些最后一次更新时间早于某给定时刻的流进行垃圾回收，清除它们的预留。请注意，边缘交换机并不会阻塞并等待调度器完成这一计算，而是起初像对待任何其他流那样对待大流。

3.8 容错 (Fault-Tolerance)

任意一对主机之间存在多条可用路径所带来的冗余，使胖树拓扑在容错方面颇具吸引力。我们提出一种简单的故障广播协议，允许交换机绕过其下游一两跳处的链路或交换机故障进行路由。在该方案中，网络中的每台交换机都与它的每个邻居维持一个双向转发检测会话（Bidirectional Forwarding Detection,

BFD），以判定链路或邻接交换机何时发生故障。从容错的角度看，有两类故障可以应对：(a) 荚舱内下层与上层交换机之间的故障；(b) 核心交换机与上层交换机之间的故障。显然，下层交换机的故障会导致与之直连的主机失去连接；在叶子处设置冗余的交换单元是容忍此类故障的唯一办法。我们这里描述链路故障，因为交换机故障会触发相同的 BFD 告警并引发相同的响应。

3.8.1 下层到上层交换机（Lower- to Upper-layer Switches）

下层与上层交换机之间的链路故障会影响三类流量：

1. 源自下层交换机的、向外的荚舱间和荚舱内流量。这种情况下，本地的流分类器将该链路的"代价"设为无穷大，不再为它分配任何新流，并选择另一台可用的上层交换机。
2. 以该上层交换机作为中间节点的荚舱内流量。作为响应，这台交换机广播一个标签（tag），通知同一荚舱中所有其他下层交换机该链路发生了故障。这些交换机在分配新流时会检查其预定输出端口是否对应于这些标签之一，并尽可能避开它（脚注：我们依赖端到端机制来重启被中断的流）。
3. 进入该上层交换机的荚舱间流量。与该上层交换机相连的核心交换机，是它访问该荚舱的唯一通道，因此该上层交换机向其所有核心交换机广播这个标签，表明它无法承载发往该下层交换机所辖子网的流量。这些核心交换机进而把这个标签镜像（mirror）给它们在其他荚舱中所连接的所有上层交换机。最终，这些上层交换机在为该子网分配新流时，会避开那台受影响的核心交换机。

3.8.2 上层到核心交换机（Upper-layer to Core Switches）

从一台上层交换机到核心的链路故障会影响两类流量：

1. 向外的荚舱间流量。
这种情况下，本地路由表将受影响的链路标记为不可用，并在本地选择另一台核心交换机。
2. 进入的荚舱间流量。这种情况下，核心交换机向它直连的所有其他上层交换机广播一个标签，表明它无法承载发往那整个荚舱的流量。如前所述，这些上层交换机在分配发往该荚舱的流时会避开那台核心交换机。

自然地，当故障的链路和交换机恢复并重新建立其 BFD

会话时，会执行上述步骤的逆操作以取消其影响。此外，将第 3.7 节的方案改造以适应链路和交换机故障也相对简单。调度器将任何被报告为宕机的链路标记为忙或不可用，从而使任何包含该链路的路径不被考虑，实际上就是让大流绕过故障进行路由。

3.9 功耗与散热问题（Power and Heat Issues）

除了性能和成本之外，数据中心设计中出现的另一个主要问题是功耗。在数据中心中构成互连较高级别的那些交换机，通常消耗数千瓦功率，而在大规模数据中心中，互连的功耗需求可达数百千瓦。几乎同等重要的是交换机的散热问题。企业级交换机产生大量热量，因而需要专用的冷却系统。

> 图 6：功耗与散热的比较。横轴为不同交换机型号，纵轴分别为每 Gbps 的功耗（Watts/Gbps）和每 Gbps 的散热（BTU/hr/Gbps）。图中显示，归一化到带宽后，10 GigE 交换机（横轴最后三个）每 Gbps 消耗约为商用 GigE 交换机两倍的功率，并散发约三倍的热量。

> 图 7：总功耗与总散热的比较。对比层级化设计（Hierarchical design）与胖树（Fat-tree）两种方案的总功耗（kW）和总散热（kBTU/hr）。

在本节中，我们分析我们架构的功耗需求和散热，并将其与其他典型方法进行比较。我们的分析基于交换机数据手册中报告的数值，尽管我们承认这些报告值是由不同厂商以不同方式测量的，因而未必总能反映部署中的系统特性。

为比较每类交换机的功耗，我们将交换机的总功耗和总散热，按该交换机能够支持的总带宽（以 Gbps 计）做归一化。图 6 绘制了三种不同交换机型号的平均值。如图所示，归一化到带宽后，10 GigE 交换机每 Gbps 消耗约两倍的瓦数，散发约三倍于商用 GigE 交换机的热量。

最后，我们还计算了一个能支持约 27k

台主机的互连的估计总功耗和总散热。对于层级化设计，我们采用 576 台 ProCurve 2900 边缘交换机和 54 台 BigIron RX-32 交换机（36 台在汇聚层，18 台在核心层）。胖树架构则采用 2880 台 Netgear GSM 7252S 交换机。我们之所以能使用更便宜的 NetGear 交换机，是因为在胖树互连中我们不需要 10 GigE 上行链路（ProCurve 中存在该上行链路）。图 7 显示，尽管我们的架构采用了更多的单台交换机，但其功耗和散热都优于当前数据中心设计所产生的水平，功耗降低 56.6%，散热降低 56.5%。当然，实际的功耗和散热必须在部署中测量；我们将这样的研究留作正在进行的工作。

4. 实现（Implementation）

为验证本文所描述的通信架构，我们构建了上一节所述转发算法的一个简单原型。我们使用 NetFPGA 完成了一个原型。NetFPGA 中含有一个利用 TCAM 的 IPv4 路由器实现。我们按照第 3.4 节的描述适当修改了路由表查找例程。我们的修改总计不到 100 行额外代码，并且没有引入任何可测量的额外查找延迟，这支持了我们的信念：我们所提出的修改可以被纳入现有交换机中。

为开展更大规模的评估，我们还使用 Click 构建了一个原型，这也是本文评估的重点。Click 是一种模块化的软件路由器架构，支持实现实验性的路由器设计。一个 Click 路由器是由若干被称为元素（element）的分组处理模块构成的图，这些元素执行诸如路由表查找或递减分组 TTL 之类的任务。当这些 Click 元素被链接在一起时，便能在软件中实现复杂的路由器功能和协议。

4.1 TwoLevelTable（两层表元素）

我们构建了一个新的 Click 元素 TwoLevelTable，它实现了第 3.3 节所述两层路由表的思想。该元素有一个输入，以及两个或更多输出。路由表的内容使用一个给出所有前缀和后缀的输入文件来初始化。对于每个分组，TwoLevelTable 元素查找最长匹配的一级前缀。如果该前缀是终止性的，它会立即在该前缀的端口上转发该分组。否则，它会在二级表上执行右对齐的最长匹配后缀查找，并在相应端口上转发。

该元素可以替换文献中所提供的、符合标准的 IP

路由器配置示例中的那个中央路由表元素。我们生成了一个类似的 4 端口版本 IP

路由器，并增加了一个修改：在所有端口上添加限带宽元素，以模拟链路饱和容量。

4.2 FlowClassifier（流分类器元素）

为提供第 3.6 节所述的流分类功能，我们描述对 Click 元素 FlowClassifier 的实现，它有一个输入和两个或更多输出。它基于入向分组的源和目的 IP 地址执行简单的流分类，使得具有相同源和目的后续分组从同一端口出去（以避免分组乱序）。该元素还有一个额外目标：最小化其负载最高与负载最低的输出端口之间总流量容量的差异。

即便事先已知各条流的大小，这个问题也是 NP 难的装箱（Bin Packing）优化问题的一个变种。然而，实际上流的大小是无法先验得知的，这使问题更加困难。我们遵循算法 3 中概述的贪心启发式方法。每隔几秒，该启发式方法会尝试在需要时切换至多三条流的输出端口，以最小化其各输出端口间总流量容量的差异。

```
// 对每个入向分组调用
1 IncomingPacket(packet)
2 begin
3   Hash source and destination IP fields of packet; // 对分组的源和目的 IP 字段做哈希
   // 我们以前见过这条流吗？
4   if seen(hash) then
5     Lookup previously assigned port x; // 查找此前为该流分配的端口 x
6     Send packet on port x; // 在端口 x 上发送（避免乱序）
7   else
8     Record the new flow f; // 记录这条新流 f
9     Assign f to the least-loaded upward port x; // 将 f 分配给负载最轻的上行端口 x
10    Send the packet on port x;
11  end
12 end

// 每隔 t 秒调用一次
13 RearrangeFlows()
14 begin
15   for i = 0 to 2 do // 最多重排 3 条流
16     Find upward ports pmax and pmin with the largest and
       smallest aggregate outgoing traffic, respectively; // 找出总出向流量最大和最小的上行端口
17     Calculate D, the difference between pmax and pmin; // 计算二者差值 D
18     Find the largest flow f assigned to port pmax whose size
       is smaller than D; // 在 pmax 上找出小于 D 的最大流 f
19     if such a flow exists then
20       Switch the output port of flow f to pmin; // 将流 f 的输出端口切换到 pmin
21     end
22   end
23 end
```

> 算法 3：流分类器的启发式方法。在第 5 节的实验中，t 取 1 秒。

回想一下，FlowClassifier 元素是两层表在流量扩散方面的替代方案。使用这些元素的网络将采用普通的路由表。例如，一台上层荚舱交换机的路由表像之前一样包含分配给该荚舱的所有子网前缀。然而，除此之外，我们添加一个 /0

前缀，以匹配所有其余需要向上均匀分散到核心层的荚舱间流量。所有仅匹配该前缀的分组都被导向 FlowClassifier 的输入。分类器按照上述启发式方法尝试将出向的荚舱间流均匀分布到它的各个输出，并且它的输出直接连接到核心交换机。核心交换机不需要分类器，它们的路由表保持不变。

请注意，这个方案具有软状态，这些状态对正确性而言并非必需，仅用作性能优化。这个分类器偶尔会带来干扰，因为可能会周期性地重排数量最少的若干流，从而潜在地导致分组乱序。然而，它也能

自适应于动态变化的流大小，并且在长期上是"公平的"（脚注：所谓公平，是指由于持续跟踪所有流的大小以逼近流到端口的最优分布，初始放置决策会不断被纠正）。

4.3 FlowScheduler（流调度器元素）

如第 3.7 节所述，我们实现了元素 FlowReporter，它驻留在所有边缘交换机中，检测大小超过某给定阈值的出向流。它定期向中央调度器发送关于这些活跃大流的通知。

FlowScheduler 元素接收来自边缘交换机的关于活跃大流的通知，并尝试为它们找到无争用的路径。为此，它保存网络中所有链路的二元状态，以及一个此前已放置流的列表。对于任何新的大流，调度器在源主机和目的主机之间所有等价路径中执行线性搜索，以找到一条其路径组件全部未被预留的路径。一旦找到这样一条路径，流调度器将所有组件链路标记为已预留，并向相关荚舱交换机发送关于此流路径的通知。我们也对荚舱交换机做了修改，使其能够处理来自调度器的这些端口重分配消息。

调度器维护两个主要数据结构：一个表示网络中所有链路的二元数组（总共 $4k(k/2)^2$ 条链路），以及一个存放此前已放置流及其所分配路径的哈希表。为新流放置而进行的线性搜索平均需要 $2 * (k/2)^2$ 次内存访问，使调度器的计算复杂度在空间上为 $O(k^3)$ 、在时间上为 $O(k^2)$ 。 k （每台交换机的端口数）的一个典型取值为 48，使这两个量都处于可管理的范围内，第 5.3 节对此做了量化。

5. 评估（Evaluation）

为测量我们设计的总双边带宽，我们生成一套通信映射的基准测试集（benchmark suite），用以评估使用 TwoLevelTable 交换机、FlowClassifier 和 FlowScheduler 的 4 端口胖树的性能。我们把这些方法与一种过载比为 3.6:1 的标准层级树进行比较，后者类似于当前数据中心设计中常见的结构。

5.1 实验描述（Experiment Description）

在 4 端口胖树中，有 16 台主机、四个荚舱（每个荚舱含四台交换机）和四台核心交换机。因此，总共有 20 台交换机和 16 台端主机（对于更大的集群，交换机数量将少于主机数量）。我们将这 36 个元素复用到十台物理机上，这些物理机由一台带有 1 千兆以太网链路的 48 端口 ProCurve 2900 交换机互连。这些机器配有主频 2.33GHz 的双核 Intel Xeon CPU，4096KB 缓存和 4GB 内存，运行 Debian GNU/Linux 2.6.17.3。每个荚舱的交换机托管在一台机器上；每个荚舱的主机托管在一台机器上；其余两台机器各运行两台核心交换机。交换机和主机都是 Click 配置，运行在用户态。网络中 Click 元素之间的所有虚拟链路都被限带宽到 96Mbit/s，以确保该配置不受 CPU 限制。

对于作为比较对象的层级树网络，我们有四台机器各运行四台主机，以及四台机器各运行四台荚舱交换机并带一条额外上行链路。这四台荚舱交换机连接到一台运行在专用机器上的 4 端口核心交换机。为在荚舱交换机到核心交换机的上行链路上强制实现 3.6:1 的过载比，这些链路被限带宽到 106.67Mbit/s，而所有其他链路被限带宽到 96Mbit/s。

每台主机产生恒定的 96Mbit/s 出向流量。我们测量它的入向流量速率。在所有双射（bijective）通信映射下，所有主机的入向流量总和的最小值，即为该网络的有效双边带宽。

5.2 基准测试集（Benchmark Suite）

我们按照以下策略生成通信对，并附加一个约束：任何主机恰好从一台主机接收流量（即该映射是一一对一的）：

- 随机（Random）：一台主机以均匀概率向网络中任意其他主机发送。
- 跨步 Stride(i)：索引为 x 的主机将向索引为 $(x + i) \bmod 16$ 的主机发送。
- 交错概率 Staggered Prob(SubnetP, PodP)：一台主机以概率 SubnetP 向其子网内的另一台主机发送，以概率 PodP 向其荚舱内发送，以概率 $1 - \text{SubnetP} - \text{PodP}$ 向其他任何主机发送。
- 荚舱间入向 Inter-pod Incoming：多个荚舱向同一荚舱内的不同主机发送，并且都恰好选择了同一台核心交换机。该核心交换机到目的荚舱的链路将被过载。这种情况下，最坏情况的局部过载比为 $(k - 1) : 1$ 。
- 同 ID 出向 Same-ID Outgoing：同一子网中的主机向网络中别处的不同主机发送，使得目的主机具有相同的主机 ID 字节。静态路由技术迫使它们采用相同的出向上行端口。这种情况下的最坏情况比为 $(k/2) : 1$ 。这是 FlowClassifier 预期能最大程度改善性能的情形。

5.3 结果（Results）

表 2 展示了上述实验的结果。这些结果是基准测试在 5 次运行/排列上、每次各 1 分钟的平均值。正如预期，对于任何全部为荚舱间的通信模式，传统树会饱和通往核心交换机的链路，因此在该情形下对所有主机仅达到约 28% 的理想带宽。通信对彼此越靠近，树的表现就越好。

> 表 2：网络的总带宽，以理想双边带宽的百分比表示，分别针对传统树（Tree）、两层表（Two-Level Table）、流分类（Flow Classification）和流调度（Flow Scheduling）四种方法。胖树网络的理想双边带宽为 1.536Gbps。

测试模式	Tree（传统树）	Two-Level（两层表）	Flow Classification（流分类）	Flow Scheduling（流调度）
---	---	---	---	---
Random（随机）	53.4%	75.0%	76.3%	93.5%
Stride(1)	100.0%	100.0%	100.0%	100.0%
Stride(2)	78.1%	100.0%	100.0%	99.5%
Stride(4)	27.9%	100.0%	100.0%	100.0%
Stride(8)	28.0%	100.0%	100.0%	99.9%
Staggered Prob(1.0, 0.0)	100.0%	100.0%	100.0%	100.0%
Staggered Prob(0.5, 0.3)	83.6%	82.0%	86.2%	93.4%
Staggered Prob(0.2, 0.3)	64.9%	75.6%	80.2%	88.5%
Inter-pod Incoming（荚舱间入向，最坏情况）	28.0%	50.6%	75.1%	99.9%

测试模式	Tree (传统树)	Two-Level (两层表)	Flow Classification (流分类)	Flow Scheduling (流调度)
Same-ID Outgoing (同 ID 出向, 最坏情况)	27.8%	38.5%	75.4%	87.4%

两层表交换机对于随机通信模式达到约 75% 的理想双边带宽。这可以用表的静态特性来解释；任意给定子网上的两台主机，有 50% 的概率向具有相同主机 ID 的主机发送，在这种情况下它们的合并吞吐量会减半，因为它们会被在同一输出端口上转发。这使得二者的期望值为 75%。我们预期，随着 k 增大，两层表在随机通信下的性能会改善，因为 k 越大，多条流在单条链路上碰撞的可能性越小。两层表的"荚舱间入向"情形给出 50% 的双边带宽；然而，"同 ID 出向"效应又因核心路由器处的拥塞而进一步加重。

由于其动态的流分配与重分配，流分类器在所有情形下都优于传统树和两层表，最坏情况的双边带宽约为 75%。然而，它仍不完美，因为它所避免的那种拥塞完全是局部的；由于在上游一两跳处所做的路由决策，仍有可能在某台核心交换机处造成拥塞。这种次优路由之所以发生，是因为交换机只拥有局部的知识。

另一方面，FlowScheduler 依据全局知识行动，尝试将大流分配到互不相交的路径上，从而对随机通信映射达到 93% 的理想双边带宽，并在所有基准测试中胜过所有其他方法。对于大型任意网络而言，使用一个掌握所有活跃大流和所有链路状态的中心化调度器或许并不可行，但胖树拓扑的规整性极大地简化了对无争用路径的搜索。

在一项单独的测试中，表 3 展示了当中央调度器运行在一台配置适中的 2.33GHz 商用 PC 上时的时间和空间需求。对于不同的 k，我们生成虚构的放置请求（每台主机一个），以测量处理一个放置请求的平均时间，以及为所维护的链路状态和流状态数据结构所需的总内存。对于一个 27k 台主机的网络，调度器仅需 5.6MB 内存，且能在 0.8ms 之内放置一条流。

> 表 3：流调度器的时间与内存需求。

```
>
> | k | 主机数 | 平均时间/请求 (μs) | 链路状态内存 | 流状态内存 |
> |---|---|---|---|---|
> | 4 | 16 | 50.9 | 64 B | 4 KB |
> | 16 | 1,024 | 55.3 | 4 KB | 205 KB |
> | 24 | 3,456 | 116.8 | 14 KB | 691 KB |
> | 32 | 8,192 | 237.6 | 33 KB | 1.64 MB |
> | 48 | 27,648 | 754.43 | 111 KB | 5.53 MB |
```

6. 封装 (Packaging)

胖树拓扑用于集群互连的一个缺点是互连所有机器所需的线缆数量。用 10 GigE 交换机执行汇聚有一个显而易见的好处：在层级结构中向上传输相同带宽量所需的线缆数量可减少 10 倍。在我们所提议的胖树拓扑中，我们既不利用 10 GigE 链路也不利用 10 GigE 交换机，一方面是因为非商用部件会抬高成本，更重要的是，因为胖树拓扑的扩展特性关键地依赖于在层级结构每一层向多台交换机的大扇出。

在承认布线开销的增加是胖树拓扑所固有的前提下，本节中我们考虑一些封装技术来缓解这种开销。总而言之，我们所提议的封装技术消除了大部分所需的外部布线，并缩短了所需线缆的总长度，这进而简化了集群管理并降低了总成本。此外，这种方法还允许网络的增量式部署（incremental deployment）。

> 图 8：所提议的封装方案。唯一的外部线缆是在各荚舱与核心节点之间。

我们在一个最大容量 27648 节点集群的背景下呈现我们的方法，该集群以 48 端口以太网交换机作为胖树的构建模块。这一设计可推广到不同规模的集群。我们从构成更大集群复制单元（replication unit）的各个荚舱的设计开始，见图 8。每个荚舱由 576 台机器和 48 台独立的 48 端口 GigE 交换机构成。为简单起见，我们假设每台端主机占用一个机架单元（1RU），且每个机架可容纳 48 台机器。因此，每个荚舱由 12 个机架构成，每个机架含 48 台机器。

我们将构成每个荚舱中胖树前两层的那 48 台交换机放置在一个集中的机架中。不过，我们假设有能力将这 48 台交换机封装成一个单一的整体单元，对外提供 1152 个面向用户的端口。我们称之为荚舱交换机（pod switch）。在这些端口中，576 个直接连接到荚舱内的机器，对应边缘处的连接。另外 576 个端口扇出，连接到构成胖树核心层的 576 台交换机各自的一个端口。请注意，以这种方式封装的 48 台交换机实际上总共有 2304 个端口（ 48×48 ）。其余的 1152 个端口在荚舱交换机内部布线，以满足荚舱边缘层与汇聚层之间所需的互连（见图 3）。

我们进一步把构成胖树顶部的、所需的那 576 台核心交换机分散到各个荚舱中。假设总共有 48 个荚舱，每个将容纳所需核心交换机中的 12 台。在从每台荚舱交换机扇出到核心的 576 条线缆中，有 12 条将直接连接到放置在同一荚舱附近的核心交换机。其余的线缆将以每组 12 条的方式扇出，连接到放置在远端荚舱中的核心交换机。请注意，线缆以每组 12 条在荚舱之间移动、以每组 48 条从机架到荚舱交换机移动这一事实，为恰当的“线缆封装（cable packaging）”以降低布线复杂度提供了额外机会。

最后，最小化线缆总长度是另一个重要的考虑因素。为此，我们将机架沿两个维度环绕荚舱交换机摆放，如图 8 所示（我们不考虑三维的数据中心布局）。这样做将相对于荚舱内各个机架更“水平”的布局缩短线缆长度。类似地，我们将各荚舱布置成一个 7×7 的网格（缺一个位置）以容纳全部 48 个荚舱。同样，这种网格布局将缩短荚舱间到相应核心交换机的布线距离，并支持对线缆长度和封装做某种标准化，以支撑荚舱间的连接。

我们还考虑过一种没有把交换机集中到中央机架的替代设计。在这种方法中，每个机架分布两台 48 端口交换机。主机以每组 24 台的方式连接到交换机。这种方法的优点是：连接主机到其第一跳交换机所需的线缆要短得多，并且如果机架内部恰当封装，还可以完全消除这些线缆。我们放弃了这种方法，因为这样会失去消除每个荚舱内那 576 条互连边缘层与汇聚层的线缆的机会。这些线缆将不得不在每个荚舱的 12

个机架之间交错穿行，增加显著的复杂度。

7. 相关工作 (Related Work)

我们在数据中心网络架构方面的工作，必然建立在若干相关领域的工作之上。与我们的努力或许最密切相关的，是构建可扩展互连的各种努力，它们主要源自超级计算机和大规模并行处理（MPP）社区。许多 MPP 互连被组织成胖树，包括来自 Thinking Machines 和 SGI 的系统。Thinking Machines 采用伪随机的转发决策，在胖树链路之间执行负载均衡。虽然这种方法能实现良好的负载均衡，但它容易出现分组乱序。Myrinet 交换机也采用胖树拓扑，在基于集群的超级计算机中很受欢迎。Myrinet 采用基于预先确定的拓扑知识的源路由（source routing），从而实现直通（cut-through）的低延迟交换机实现。主机也负责通过测量往返延迟在可用路由之间进行负载均衡。相对于所有这些努力，我们聚焦于利用商用以太网交换机来互连大规模集群，展示了恰当的路由与封装技术。

InfiniBand 是高性能计算环境中一种流行的互连，目前正在向数据中心环境迁移。InfiniBand 也使用 Clos 拓扑的变体来实现可扩展带宽。例如，Sun 最近宣布了一款 3456 端口的 InfiniBand 交换机，它由 720 台 24 端口的 InfiniBand 交换机以 5 级胖树排列而成。然而，InfiniBand 强加了它自己的第 1–4 层协议，使得以太网/IP/TCP 在某些场景中更具吸引力，尤其是随着 10Gbps 以太网价格的持续下降。

另一种流行的 MPP 互连拓扑是环面（Torus），例如在 BlueGene/L 和 Cray XT3 中。环面在一个 k 维点阵中将一个处理器与其若干邻居直接互连。维数决定了源与目的之间的期望跳数。在 MPP 环境中，环面的好处是不需要任何专用交换单元，以及电气上更简单的点对点链路。在集群环境中，环面的布线复杂度很快变得难以承受，并且将所有路由和转发功能卸载到商用主机/操作系统上通常是不切实际的。

我们所提议的转发技术与诸如 OSPF2 和等价多路径（ECMP）之类的现有路由技术相关。我们的多路径方案利用了胖树拓扑的特定属性来实现良好的性能。相对于我们的工作，ECMP 提出了三类无状态转发算法：(i) 轮询和随机化；(ii) 区域分裂（region splitting），即将一个特定前缀分裂为两个掩码长度更大的前缀；(iii) 一种哈希技术，基于源和目的地址在一组输出端口之间分摊各条流。第一种方法存在潜在的分组乱序问题，对 TCP 尤其有害。第二种方法可能导致路由前缀数量的爆炸式增长。在一个 25000 台主机的网络中，这将需要约 600000 条路由表条目。除了增加成本外，这种规模的表查找将带来显著延迟。出于这个原因，当前企业级路由器允许最多 16 路 ECMP 路由。最后一种方法在做分配决策时不考虑流的带宽，这可能很快导致即便对简单通信模式也出现过载。

8. 结论 (Conclusions)

带宽日益成为大规模集群中的可扩展性瓶颈。解决这一瓶颈的现有方案，都围绕交换机的层级结构展开，在层级结构顶部使用昂贵的、非商用的交换机。在任意给定时间点，高端交换机的端口密度都会限制整体集群规模，同时还会带来高昂成本。在本文中，我们提出了一种数据中心通信架构，它利用商用以太网交换机为大规模集群提供可扩展带宽。我们以胖树为基础构建拓扑，然后提出在保持与以太网、IP 和 TCP 向后兼容的同时执行可扩展路由的技术。

总体而言，我们发现自己能够以显著低于现有技术的成本提供可扩展带宽。尽管要充分验证我们的方法还需要更多工作，但我们相信，大量的商用交换机有潜力在数据中心中取代高端交换机，正如商用 PC 集群在高端计算环境中取代了超级计算机一样。

致谢 (Acknowledgments)

我们感谢 George Varghese 以及匿名审稿人对本文早期草稿提供的宝贵反馈意见。

9. 参考文献 (References)

(参考文献略)